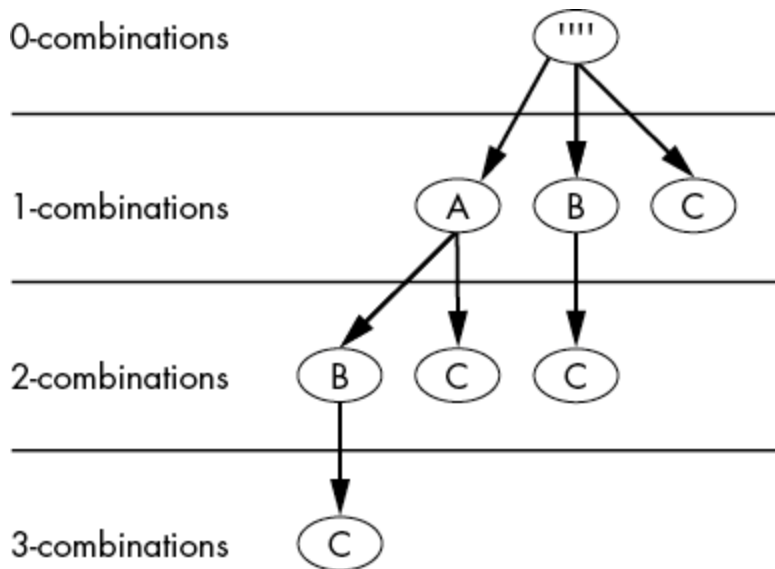


- The 1-combination D is the mirror of the 3-combination ABC.

We'll create a function called `getCombos()` that takes two arguments: a `chars` string with the letters to get combinations from, and the size of the combinations `k`. The return value is an array of strings of combinations from the string `chars`, each of length `k`.

We'll use a head-tail technique with the `chars` argument. For example, say we call `getCombos('ABC', 2)` to get all the 2-combinations from {A, B, C}. The function will set A as the head and BC as the tail. [Figure 6-5](#) shows the tree for selecting 2-combinations from {A, B, C}.



*Figure 6-5: Tree showing every possible 2-combination from the set {A, B, C}*

Let's ask our three recursive algorithm questions:

**What is the base case?** The first base case is a `k` argument of 0, meaning that a 0-combination is requested, which is always an array of the blank string no matter what `chars` is. The second case occurs if `chars` is the blank string, which is an empty array since no possible combinations can be made from a blank string.

**What argument is passed to the recursive function call?** For the first recursive call, the tail of `chars` and `k - 1` are passed. For the second recursive call, the tail of `chars` and `k` are passed.